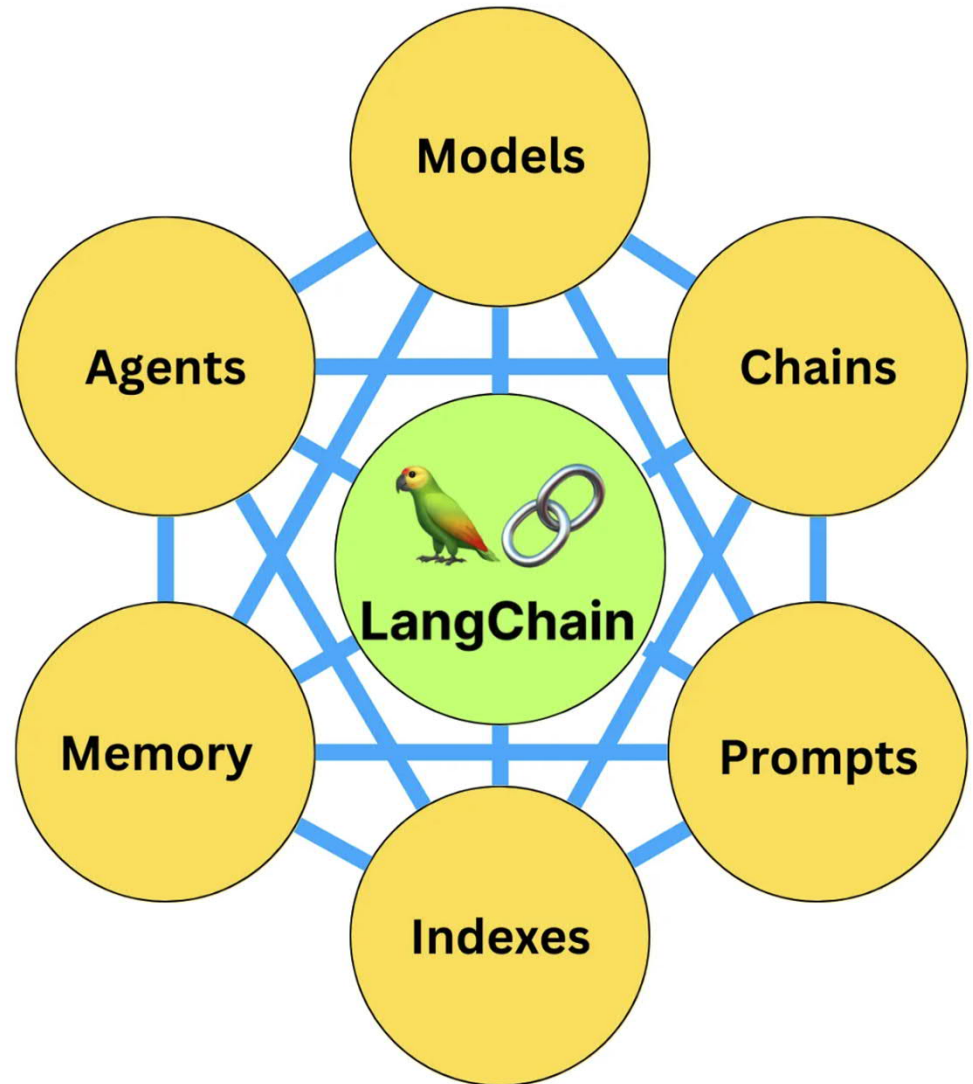


Langchain Agents

CS5305: Scalable AI Services and Agentic AI

Langchain Components



Chains

- A Chain is a sequence of steps that takes input → processes it → produces output.
 - Each step can be a prompt, a model call, or a function.
- Characteristics:
 - Predefined workflow.
 - Deterministic: always follows the same sequence.
 - Usually one main task (e.g., summarization, question answering).
 - Can be LCEL pipeline:
 - template | model | parser.

Agents

- An Agent is a higher-level system that can decide dynamically which actions or tools to use based on the input.
- It can call multiple chains, tools, APIs, or even run Python code, and decide the order at runtime.
- Characteristics:
 - Flexible and intelligent: chooses actions dynamically.
 - Can use tools (search engines, calculators, APIs, custom functions).
 - Good for complex tasks or multi-step problem solving.
 - Uses LLM reasoning + tools to decide next step.

Agents

- Langchain Agents:
 - Use an LLM Model to decide actions
 - The model determines which tool to call, what arguments to pass, and when to stop.
 - String (gpt-4o-mini), Object (ChatAzureOpenAI), etc.
 - Run in a loop until completion
 - An agent calls a **tool** → observes result → reasons again → repeats, until a final answer or iteration limit is reached
 - Tools → Functions/API Calls
 - Execute on a LangGraph runtime
 - Agents are created via **create_agent**
 - Compiled into a **graph-based execution engine**
 - **Nodes** = model, tools, middleware;
 - **Edges** determine flow

Agent Example

create_agent is a LangChain function that builds an AI agent which can think, choose the right tool for a task, use it, and continue reasoning until it produces a final answer.

The docstring is important—the agent reads it to decide what and how to call.

```
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import AzureChatOpenAI

@tool
def add(a: int, b: int) -> int:
    """ adds two numbers """
    return a + b

tools = [add]
model = <Invoke method to get a new AzureChatOpenAI Object>
agent = create_agent(model, tools)

result = agent.invoke({
    "messages": [{"role": "user", "content": "What is 12 + 7?"}]
})

print(result["messages"][-1].content)
```

@tool Decorator

- LangChain needs a structured way to tell the model:
 - What a function does
 - When it should be used
 - What inputs it expects
- When you add @tool to a function, LangChain:
 - Registers the function as a Tool
 - Uses type hints to infer an input schema (JSON schema)
 - Uses the docstring as the tool description shown to the LLM
 - Exposes the function so agents can call it when reasoning

@tool

- Can provide a more descriptive name in the @tool decorator

```
@tool ("add_two_numbers")  
def method_1(a: int, b: int) -> int:  
    """ adds two numbers """  
    return a + b
```

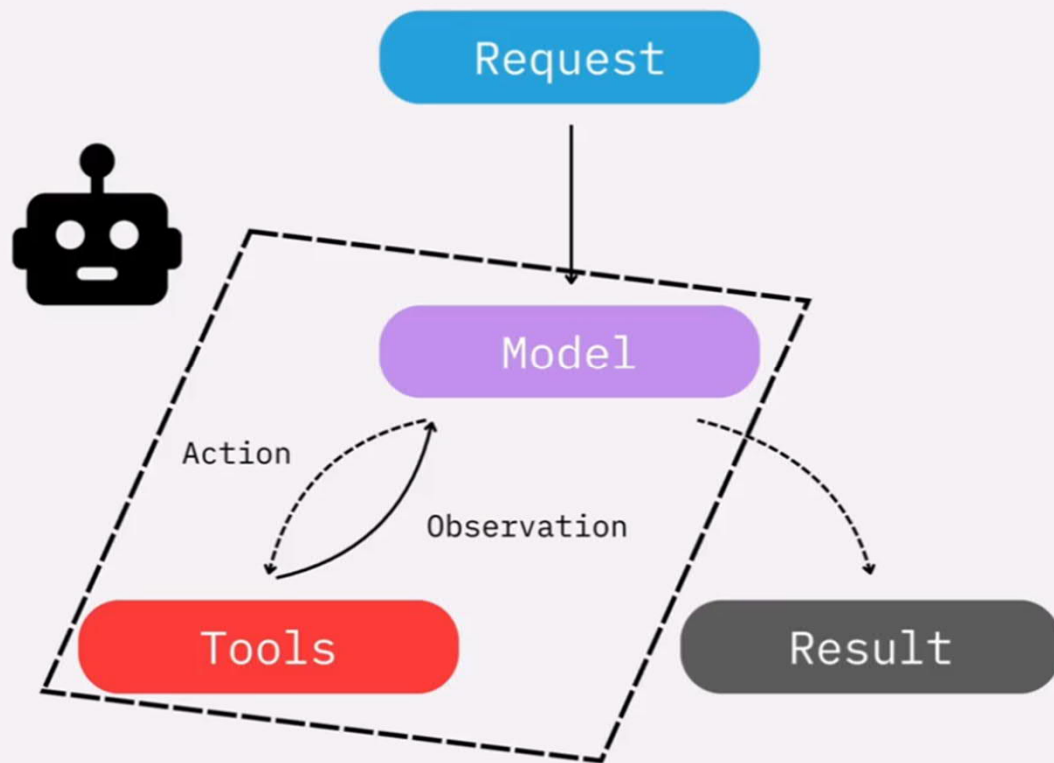
- Can update docstring as well

```
@tool ("add_two_numbers", description="add two integers")  
def method_1(a: int, b: int) -> int:  
    """ adds two numbers """  
    return a + b
```

- `add_two_numbers.invoke({a=10, b=5})`

ReAct (Reason + Act)

ReAct Agents



ReACT (Reason + Act)

- **ReAct pattern** is an **execution loop**
 - Reason → Act → Observe → (repeat) → Final Answer
- @tool + an agent that reasons about when to call that tool → ReAct
- tool defines the “**Act**” **capability** in that loop.
 - @tool → “*This is something the agent can do*”
 - Agent (LLM) → decides **when** to do it
 - Framework → executes the function and returns the **Observation**

ReAct Example

- Thought: I should look up the user's invoice total.
- Action: `query_sql`
- Action Input: `SELECT SUM(total) FROM invoices WHERE customer_id=123;`
- Observation: 15430.25
- Thought: Now I can compute the discount.
- Action: `calculator`
- Action Input: `15430.25 * 0.1`
- Observation: 1543.025
- Final Answer: The 10% discount is 1,543.03.

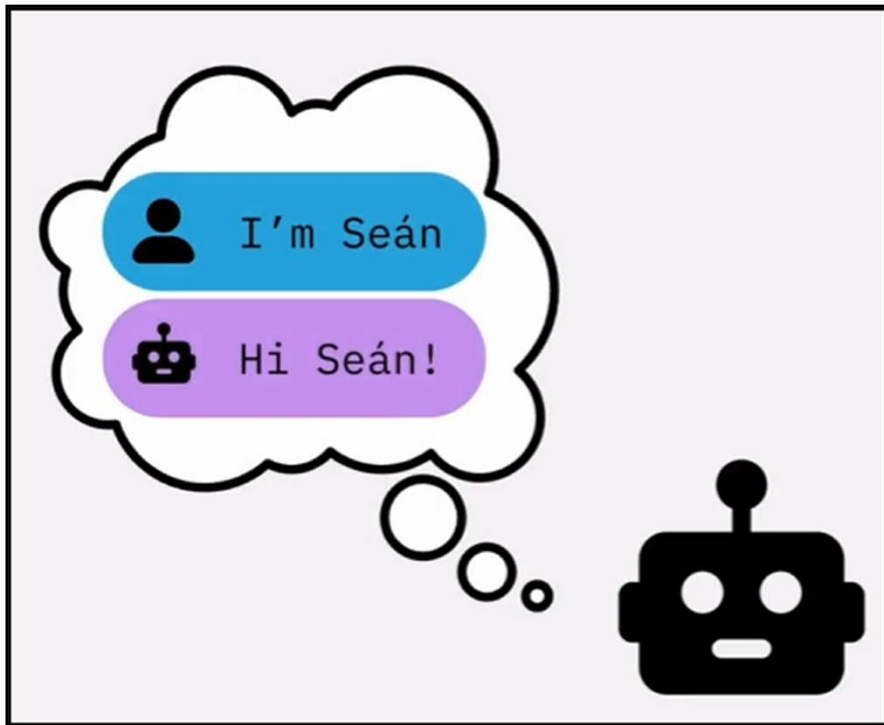
Tool Invocation (llm → agent → tool)

- When we define a tool in LangChain, LangChain uses **Pydantic** or reflection to inspect your code. It extracts:
 - **The Function Name:** (e.g., `get_weather`)
 - **The Docstring:** This becomes the "description" that tells the LLM *when* to use it.
 - **Type Hints:** These define the "parameters" (e.g., `location: str`).
- LangChain then converts the above information into a JSON schema, which is the "metadata" format that model providers (like OpenAI, Anthropic, or Google) expect.
- **create_agent** provides the complete glue (i.e., binding) under the hood:
 - It iterates through your tools list and converts each one into the JSON schema format the LLM expects.
 - Configures llm with the metadata

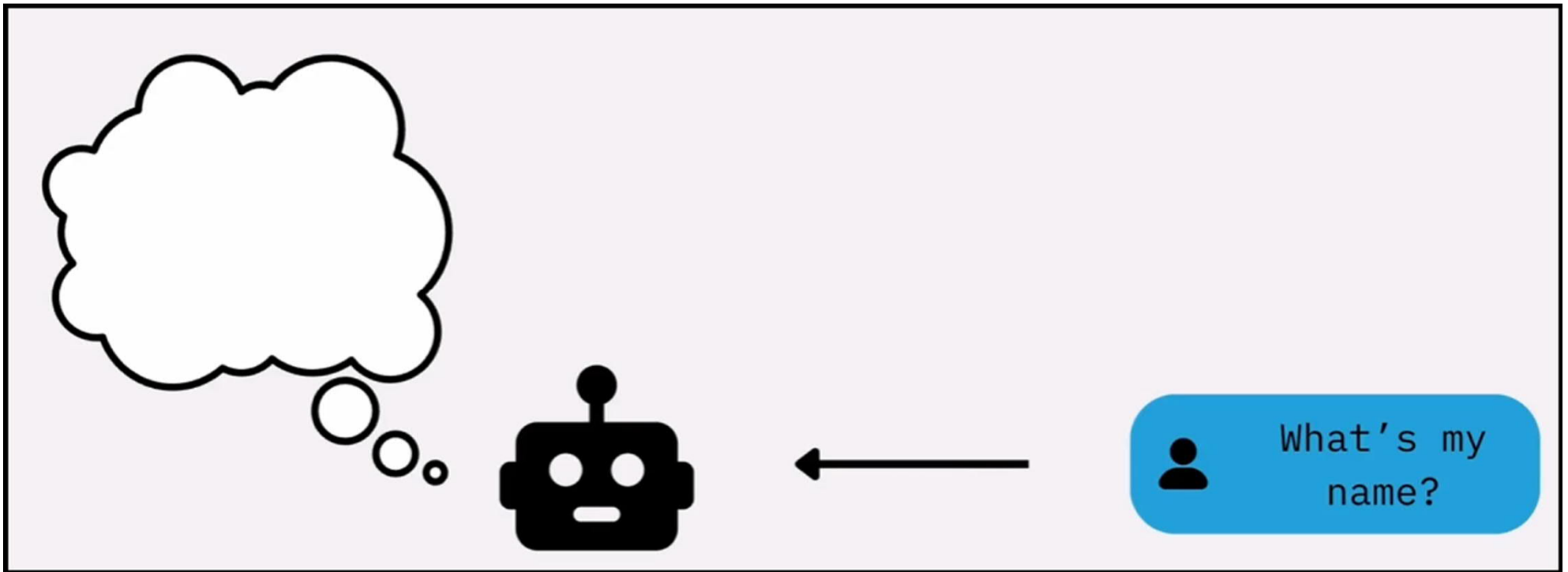


Memory

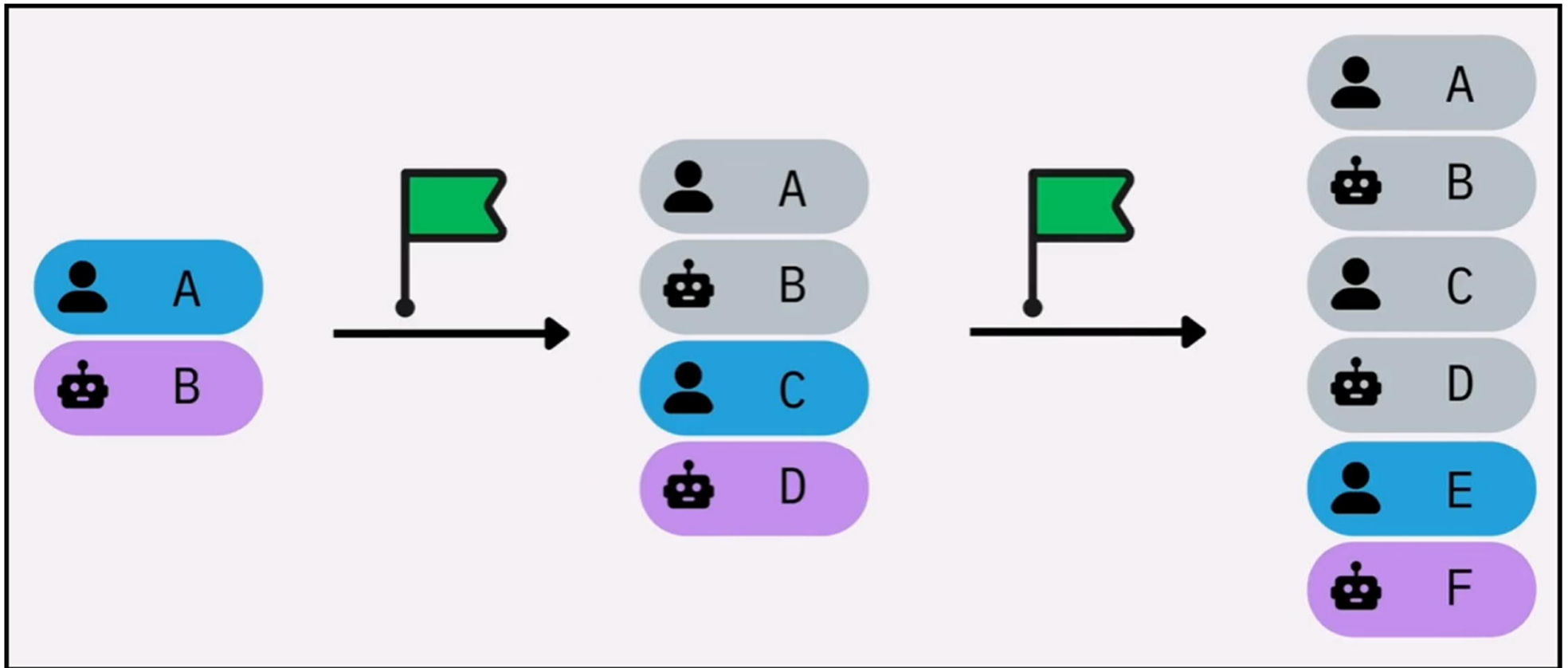
Memory



LLM/Agent has no idea!



We want LLM to “memorize conversation”

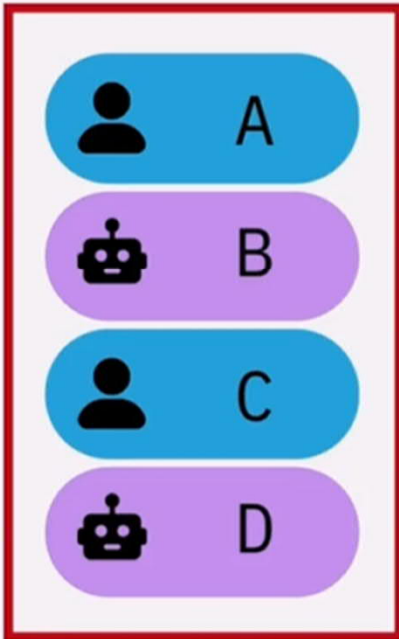


Checkpoint

- **Checkpoint** is a **persistence layer** that automatically **saves the state of a graph after each execution step** (called a *superstep*)
 - Responsible for "saving" the state of a thread so your agent can remember past interactions in a conversation
- In-Memory:
 - InMemorySaver
- Database-Backed Savers
 - PostgresSaver
 - SqliteSaver
 - MongoDBSaver
 - Etc.

Messages Only (by default)

Messages



Custom
Fields

